



more lecture notes
mhe authored 1 month ago

8ff9790f

 **README.md** 297 Bytes

Implementing a small imperative language

1. [Video lectures](#)
2. [Overview](#)
3. [Concrete syntax](#)
4. [Abstract syntax](#)
5. [Parser](#)
6. [Interpreter](#)
7. [Command line interface](#)

Overview.md 1.91 KB

Implementing a small imperative language

A video on this section can be found [here](#).

We implement an [interpreter](#) for a small imperative programming language.

Example

This is the sample file [fibonacci.xy](#):

```
{
  y := 0;
  z := 1;
  while (x > 0)
  {
    x := x - 1;
    t := y + z;
    y := z;
    z := t;
  }
}
```

input x
output y

x	11
y	0
z	1
t	-

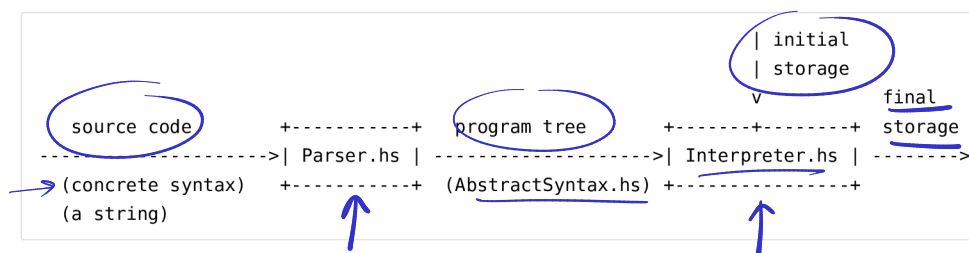
There are no IO commands in our language. The input is in the variable `x` and the output is in the variable `y`. Hence we call our language `xy`. We run this as follows:

```
$ runhaskell Runxy.hs fibonacci.xy 11
89
```

Overall structure

Given a program source code, that is, a `String`, in *concrete syntax* as above:

- We parse it, to produce a program tree in [abstract syntax](#).
- Given this and an initial storage, that is, an assignment of values to program variables, we run the program to get a new storage.



We use a program [Runxy.hs](#) to read a file into a `String` and read a value `x` from the command line arguments, which then calls the parser and then the interpreter, with an initial storage assigning the value `x` to the program variable `"x"` and finally prints the value of the variable `"y"`, as in the above example.

Next: [Concrete syntax](#)

ConcreteSyntax.md 1.29 KB

Concrete syntax

A video on this section can be found [here](#).

Here is a grammar for our language, written in [BNF](#), where expressions with lower numbers describe operators with lower precedence. The grammar is [ambiguous](#) regarding `else`. We resolve the ambiguity by associating `else`'s to the closest `if`'s to the left.

```
Program ::= Identifier := Expr;
         | { [Program] }
         | while (Expr) Program
         | if (Expr) Program
         | if (Expr) Program else Program
```

iden := Expr; { [Program] }

```
Expr ::= Expr1 | Expr1 OrOp Expr
Expr1 ::= Expr2 | Expr2 AndOp Expr1
Expr2 ::= Expr3 | Expr3 EqOp Expr2
Expr3 ::= Expr4 | Expr4 CompOp Expr3
Expr4 ::= Expr5 | Expr5 AddOp Expr4
Expr5 ::= Expr6 | Expr6 MulOp Expr5
Expr6 ::= Expr7 | NotOp Expr6
Expr7 ::= Constant | Identifier | (Expr)
```

```
OrOp ::= ||
AndOp ::= &&
EqOp ::= ==
CompOp ::= <= | < | >= | >
AddOp ::= + | -
MulOp ::= * | / | %
NotOp ::= !
```

We will use monadic parsing (please read chapter 13 of the book) to convert from concrete syntax to [abstract syntax](#).

Next: [Abstract syntax](#)

AbstractSyntax.md 1.45 KB

Abstract syntax

A video on this section can be found [here](#).

The operations are listed in the order of precedence as in the languages C and C++ (from low to high), where all the operations in the same line have the same precedence, but this is important for [parsing](#) only:

```
module AbstractSyntax where
```

```
type Identifier = String
```

```
data OpName = Or
```

```
  | And  
  | Eq  
  | Leq | Less | Geq | Greater  
  | Add | Sub  
  | Mul | Div | Mod  
  | Not
```

```
  deriving (Show)
```

```
-- ||  
-- &&  
-- ==  
-- <= < >= >  
-- + -  
-- * / %  
-- !
```

```
data Expr = Constant Integer
```

```
  | Var Identifier  
  | Op OpName [Expr]  
  deriving (Show)
```

```
data Program = Identifier := Expr
```

```
  | Block [Program]
```

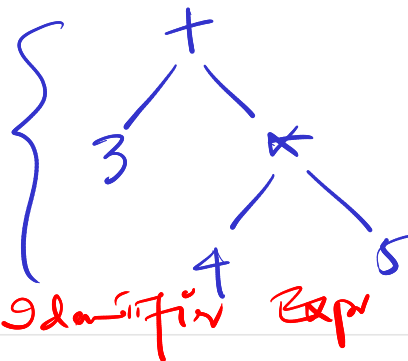
```
  | While Expr Program
```

```
  | If Expr Program
```

```
  | IfElse Expr Program Program
```

```
  deriving (Show)
```

3 + 4 * 5



Assign Identifier Expr

Notice that we are using a constructor `:=` in the `Program` type, written in infix notation. We use [monadic parsing](#) to convert from [concrete syntax](#) to abstract syntax.

Next: [Parser](#)

update links
mhe authored 1 month ago

d64c9830

Parser.md 6.47 KB

Parser

A video on this section can be found [here](#).

Recall our concrete syntax in BNF notation:

```
Program ::= Identifier := Expr;           -- assignment
          | { [Program] }                -- block
          | while (Expr) Program          -- whileStatement
          | If (Expr) Program             -- ifStatement
          | If (Expr) Program else Program

Expr ::= Expr1 | Expr1 OrOp Expr         -- lowest precedence
Expr1 ::= Expr2 (Expr2 AndOp Expr1
Expr2 ::= Expr3 | Expr3 EqOp Expr2
Expr3 ::= Expr4 | Expr4 CompOp Expr3
Expr4 ::= Expr5 | Expr5 AddOp Expr4
Expr5 ::= Expr6 | Expr6 MulOp Expr5
Expr6 ::= Expr7 | NotOp Expr6
Expr7 ::= Constant | Identifier | (Expr) -- highest precedence

OrOp  ::= ||
AndOp  ::= &&
EqOp   ::= ==
CompOp ::= <= | < | >= | >
AddOp  ::= + | -
MulOp  ::= * | / | %
NotOp  ::= !
```

The following is a direct translation of BNF to monadic [parsing combinators](#):

```
module Parser where

import Data.Char
import Control.Monad

import AbstractSyntax
import Parsing
```

The function `program` parses a program according to the above BNF definition. The result is a `Program` tree in the `Parser` monad:

```
program :: Parser Program
```

Similarly, the various `expr` functions below parse expressions, with an expression tree in the `Parser` monad:

```
expr, expr1, expr2, expr3, expr4, expr5, expr6, expr7 :: Parser Expr
```

And the following `parse` and `return Expr` constructors:

```
orOp, andOp, eqOp, compOp, addOp, mulOp, notOp :: Parser ([Expr] -> Expr)
```

Based on the BNF production rule for programs, we define:

```
program =
  assignment
  <|> block
  <|> whileStatement
  <|> ifStatement
```

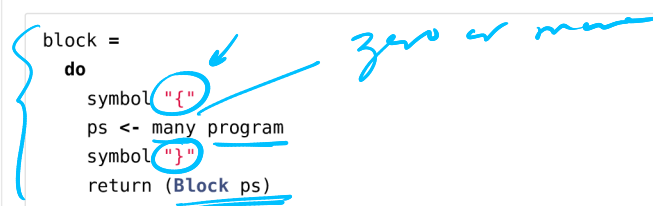
where the functions `assignment`, `block`, `whileStatement` and `ifStatement` are defined below. The production rule for assignments is `Identifier := Expr;`, which we translate as follows to monadic parsing:



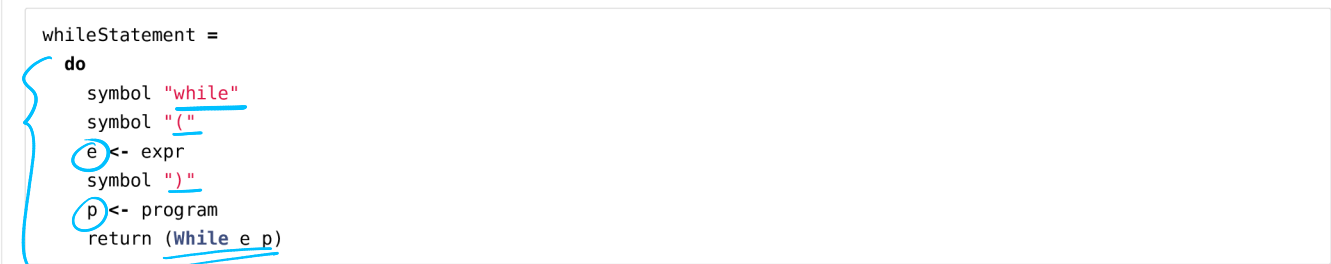
This works as follows:

- We parse an identifier with `identif` (defined below), which is bound to the variable `i`.
- We then parse the string `":="`.
- We then parse an expression with `expr`, which is bound to the variable `e`.
- We then parse the string `";"`.
- We finally return the program tree `i := e`.

The production rule for blocks is `{ [Program] }` (a list of programs enclosed in curly braces):



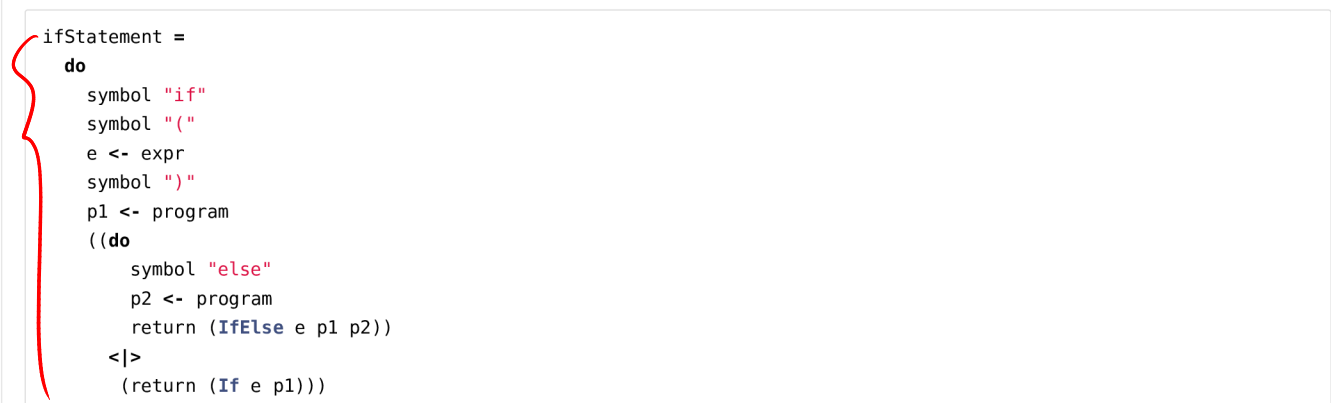
The function `many`, applied to the function `program`, parses a list of programs. It is predefined in the type class `Alternative` (see the [monadic parsing](#) lecture notes, the textbook or [hoogle](#)). This works because the type constructor `Parser` is in the `Alternative` type class. The production rule for while-statements is `while (Expr) Program`:



The production rule for if-statements is

```
If (Expr) Program
| If (Expr) Program else Program
```

This becomes, in factorized form:



That is:

- Parse `"if"` and then `"("` and then an expression `e` and then `)"` and then a program `p1`.
- Then parse one of
 - `"else"` and then a program `p2` (then return the tree `IfElse e p1 p2`), or
 - nothing (then return the tree `If e p1`).

The following function is used in order to implement parsing of BNF definitions of the form

→ $\text{expr} = \text{expr}' \mid \text{expr}' \text{ op } \text{expr}$

in factorized form:

→ $\text{binExpr} :: \text{Parser } e \rightarrow \text{Parser } ([e] \rightarrow e) \rightarrow \text{Parser } e \rightarrow \text{Parser } e$
 $\text{binExpr expr}' \text{ op expr} =$

```
do
  e' <- expr'
  ((do
    → o <- op
    → e <- expr
    return (o [e',e]))
  <|>
  → return e')
```

Handwritten diagram showing the structure of the binExpr function: $\text{op} \text{ [e', e] }$ with arrows pointing to op , expr' , and expr .

Then the definitions of expressions become:

→ $\text{expr} = \text{binExpr expr1 orOp expr}$
 $\text{expr1} = \text{binExpr expr2 andOp expr1}$
 $\text{expr2} = \text{binExpr expr3 eqOp expr2}$
 $\text{expr3} = \text{binExpr expr4 compOp expr3}$
 $\text{expr4} = \text{binExpr expr5 addOp expr4}$
 $\text{expr5} = \text{binExpr expr6 mulOp expr5}$

```
expr6 = expr7
<|>
do
  op <- notOp
  e <- expr6
  return (op [e])
```

```
expr7 = constant
<|> do
  i <- identif
  return (Var i)
<|> do
  symbol "("
  e <- expr
  symbol ")"
  return e
```

The above use the following helper functions:

```

parseOp :: String -> OpName -> Parser ([Expr] -> Expr)
parseOp s op = do
    symbol s
    return (Op op)

orOp = parseOp "||" Or

andOp = parseOp "&&" And

eqOp = parseOp "==" Eq

{
compOp = parseOp "<=" Leq
  <|> parseOp "<" Less
  <|> parseOp ">=" Geq
  <|> parseOp ">" Greater
}

{
addOp = parseOp "+" Add
  <|> parseOp "-" Sub
}

{
mulOp = parseOp "*" Mul
  <|> parseOp "/" Div
  <|> parseOp "%" Mod
}

notOp = parseOp "!" Not

```

Handwritten notes: "op And" with an arrow pointing to the 'And' in the andOp definition.

This is to parse numerical constants:

```

constant :: Parser Expr
constant = do
    n <- integer
    return (Constant(toInteger n))

```

Notice that `integer` (defined in the textbook code [Parsing.hs](#)) parses an `Int` rather than an `Integer`, and this is why we need the type cast `toInteger`.

This is to parse identifiers, which are defined like in the textbook but excluding our keywords:

```

keywords = ["if", "else", "while"]

identif :: Parser String
identif =
    do
        cs <- token identifier
        guard (not (elem cs keywords))
        return cs

```

Handwritten notes: Red circles around 'token identifier' and 'guard'.

Parsing a program can cause an error:

```

parseProgram :: String -> Program
parseProgram xs = case parse program xs of
    [(p, [])] -> p
    [(_, s)] -> error ("syntax: unparsed string " ++ s)
    [-] -> error "syntax: failed to parse program"

```

Handwritten notes: Red arrows pointing to 'parse program' and 'xs'. Blue circles around 'p' and 's'. A blue bracket on the right side of the case expression.

There is no error when we get a singleton list (unambiguous parsing) consisting of a pair $(p, [])$ where p is a program syntax tree and where $[]$ indicates that there was nothing left unparsed after the program.

Next: [Interpreter](#)

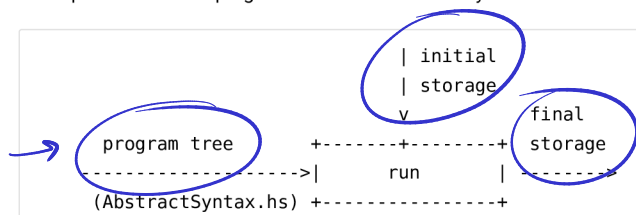
Interpreter.md 10 KB

Interpreter

A video on this section can be found [here](#).

```
module Interpreter where
import AbstractSyntax
```

We explain how to run programs written in abstract syntax:



Association List

$[("x", 11), ("y", 0), ("z", 1)]$

Representation of the storage

There are several ways to represent the storage:

1. As a list $m :: [(Identifier, Integer)]$ describing a table associating integers to identifiers.

- In this case, to find the value of an identifier i , we use the [lookup](#) function from the prelude:

```
lookup i m
```

- To update the storage m means to define a new list m' from the list m .
- Unused program variables are simply not listed in m .

2. Again as a table but using hashing, or a tree or other efficient data structures.

3. As a function $m :: Identifier \rightarrow Integer$, again associating integers to identifiers.

- In this case, to find the value of an identifier, we apply the function to the identifier.

```
m i
```

- To update the storage m means to define a new function m' from the function m .
- Unused program variables are given the value `undefined` or `error`.

Handwritten blue notes illustrating the list representation of storage:

$$\begin{array}{lcl} m & "x" & = 11 \\ m & "y" & = 0 \\ m & "z" & = 1 \\ & \vdots & \end{array}$$

Here we adopt option 3:

```
type Storage = Identifier -> Integer
```

Empty storage

At the beginning, no storage location is initialized:

```
emptyStorage :: Storage
emptyStorage i = error ("Uninitialized variable " ++ i)
```

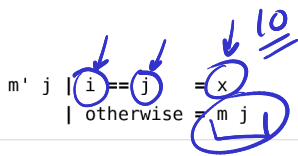
Updating the storage

To update a variable named i to the value x in $m :: Storage$ means to produce a new $m' :: Storage$ with $m' i = x$ and $m' j = m j$ for j distinct from i :

```
update :: Identifier -> Integer -> Storage -> Storage
update i x m = m'
where
  m' :: Storage
```

Handwritten blue notes illustrating the update function:

- Arrows point from x and 10 to the x and m in `update i x m`.
- The `update` function signature is underlined.
- The `m` in `update i x m` is circled in blue.



Booleans are represented as integers

We convert between numbers and booleans, so we only have the type of integers in our little language, like in the programming language C :

```
number :: Bool -> Integer
number False = 0
number True  = 1

boolean :: Integer -> Bool
boolean 0 = False
boolean _ = True
```

Remark:

Notice that the above two functions don't form an isomorphism. We only have the equation $\text{boolean}(\text{number } b) = b$ for all $b :: \text{Bool}$, or $\text{boolean} \circ \text{number} = \text{id}$ where id is the identity function defined in the prelude, but not the other way round. Technically, one says that the above two functions exhibit the type Bool as a *retract* of the type Integer , with the function boolean the *retraction* and the function number the *section*. Then the type Bool is isomorphic to the set of *fixed points* of the function $\text{number} \circ \text{boolean}$, namely 0 and 1 . See also the [section on retracts in the lecture on data types](#).

Evaluating expressions

We aim to evaluate expressions relative to a given storage:

An expression is either

- a constant (an integer), or
- a variable, or
- an operator together with a list of sub-expressions.

To evaluate, relative to storage $m :: \text{Storage}$,

- a **constant** means simply to extract the integer from it.
- a **variable**, say, "x", simply means to look up the value, via $m \text{ "x"}$.
- an **operator** and its sub-expressions, means to apply a Haskell operation associated to the operator to the evaluated sub-expressions. This is done in the following Haskell code, assuming that we evaluate operators through the auxiliary function `opEval` discussed below:

```
eval :: Storage -> Expr -> Integer
eval m (Constant x) = x
eval m (Var i)       = m i
eval m (Op o es)     = opEval o [eval m e | e <- es]
```

We look in detail at the last pattern, evaluating an operator o with sub-expressions es (a list of expressions, $[\text{Expr}]$).

Given an `OpName` (defined in the module [AbstractSyntax](#)) and the correct number of arguments (one or two here), we evaluate it:

```
opEval :: OpName -> [Integer] -> Integer
opEval Add    [x, y] = x + y
opEval Sub    [x, y] = x - y
opEval Mul    [x, y] = x * y
opEval Div    [x, y] = x `div` y
opEval Mod    [x, y] = x `mod` y
opEval Eq     [x, y] = number(x == y)
opEval Leq    [x, y] = number(x <= y)
opEval Less   [x, y] = number(x < y)
opEval Geq    [x, y] = number(x >= y)
opEval Greater [x, y] = number(x > y)
opEval And    [x, y] = number(boolean x && boolean y)
opEval Or     [x, y] = number(boolean x || boolean y)
opEval Not    [x]   = number(not(boolean x))
opEval op     xs    = error ("Interpreter bug. "
    ++ "Please contact the software maintainer. "
    ++ "Tried to apply " ++ show op
    ++ " to " ++ show xs)
```

Running programs

A video on this section can be found [here](#)

When we run a program we start with an initial storage and, if the program terminates, we get a final storage:

$\text{run} :: \text{Program} \rightarrow \text{Storage} \rightarrow \text{Storage}$

The assignment statement evaluates the expression e and stores it in the identifier i of the storage m :

$\text{run } (i := e) \ m = \text{update } i \ (\text{eval } m \ e) \ m$

To run an if-else-statement, we evaluate the expression e , and if its boolean value is true, we run the then branch p and otherwise we run the else branch q , with the given storage m :

$\text{run } (\text{IfElse } e \ p \ q) \ m$
| $\text{boolean}(\text{eval } m \ e) = \text{run } p \ m$
| otherwise = $\text{run } q \ m$

Similarly, to run an if-statement, we evaluate the expression e , and if its boolean value is true, we run the then branch with the given storage m , and otherwise we leave the given storage unchanged:

$\text{run } (\text{If } e \ p) \ m$
| $\text{boolean}(\text{eval } m \ e) = \text{run } p \ m$
| otherwise = m

To run a while-statement, we evaluate the condition e . If it is false, we leave the given storage m unchanged. If it is true, we run p on the storage m to get a new storage m' , which we use to recursively run the while-statement, to get a storage m'' , which is the result, if this recursion terminates (it need not to):

$\text{run } (\text{While } e \ p) \ m$
→ | $\text{boolean}(\text{eval } m \ e) = m''$
→ | otherwise = m
where
→ $m' = \text{run } p \ m$
→ $m'' = \text{run } (\text{While } e \ p) \ m'$

execute the body of loop
skip the loop

To run a block consisting of n program statements, starting from a storage m_1 , we apply the first statement to get m_2 , the second to get m_3 , and so on, until we apply the last statement to get the final storage m_n . We express this recursively in the following definition, where the base case is when we have zero programs, in which no transformation to the storage takes place:

$\text{run } (\text{Block } []) \ m = m$
 $\text{run } (\text{Block } (p : ps)) \ m = m''$
where
 $m' = \text{run } p \ m$
 $m'' = \text{run } (\text{Block } ps) \ m'$

An alternative approach to running programs

This approach shows that imperative programming is functional programming.

Now we define an alternative variant of the function $\text{run} :: \text{Program} \rightarrow \text{Storage} \rightarrow \text{Storage}$, using a different perspective. Specifically, we think of a program as a **storage transformer**, i.e., a map $\text{Storage} \rightarrow \text{Storage}$. Every program construction corresponds to a function that returns a storage transformer, i.e., that returns a function $\text{Storage} \rightarrow \text{Storage}$.

Firstly, the assignment statement is a storage transformer with two parameters i and e :

$\text{assign} :: \text{Identifier} \rightarrow \text{Expr} \rightarrow \text{Storage} \rightarrow \text{Storage}$
 $\text{assign } i \ e = \lambda m \rightarrow \text{update } i \ (\text{eval } m \ e) \ m$

Next, the if-else-statement takes three functions as arguments, and produces one function as the result. The first argument is a property p of the storage. The second and third arguments are two storage transformers, and the result is a storage transformer.

$\text{ifElse} :: (\text{Storage} \rightarrow \text{Bool}) \rightarrow (\text{Storage} \rightarrow \text{Storage}) \rightarrow (\text{Storage} \rightarrow \text{Storage}) \rightarrow (\text{Storage} \rightarrow \text{Storage})$
 $\text{ifElse } p \ f \ g = h$
where

```
h m = if p m then f m else g m
```

Although we use `Storage` for interpreting imperative programs, this function makes sense for any type `s` :

```
ifElse :: (s -> Bool) -> (s -> s) -> (s -> s) -> (s -> s)
ifElse p f g = h
  where
    h m = if p m then f m else g m
```

Similarly, the functions below are defined with an arbitrary type `s` rather than the specific type `Storage` .

We don't need to consider an if-statement, because `ifElse p f id` does the job, where `id` is the identity function, defined in the prelude.

Next, the while-statement takes a property `p` of states, and a storage transformer `f` . The result is a storage transformer `g` that repeatedly applies `f` to a given storage `m` until it fails to satisfy `p` :

```
while :: (s -> Bool) -> (s -> s) -> (s -> s)
while p f = g
  where
    g m = if p m then g(f m) else m
```

The block-statement is simply the composition of finitely many functions: first apply the first function to the given storage, then the second function, etc., and finally the last function. For example, `block [f,g,h] m = h(g(f m))` :

```
block :: [s -> s] -> s -> s
block []      m = m
block (f:fs) m = block fs (f m)
```

Boolean expressions represent predicates of the storage:

```
booleanValue :: Expr -> (Storage -> Bool)
booleanValue e = \m -> boolean(eval m e)
```

Finally, with these definitions, the function `run` defined above is equivalent to the following:

```
run' :: Program -> Storage -> Storage
run' (i := e)      = assign i e
run' (IfElse e p q) = ifElse (booleanValue e) (run' p) (run' q)
run' (If e p)       = ifElse (booleanValue e) (run' p) id
run' (While e p)    = while (booleanValue e) (run' p)
run' (Block ps)     = block (map run' ps)
```

Next: The main program [Runxy](#)

more lecture notes
mhe authored 1 month ago

8ff9790f

Runxy.md 2.32 KB

Runxy

A video on this section can be found [here](#).

This command line interface allows to run e.g.

\$ runhaskell Runxy.hs factorial.xy 5
120

for the file [factorial.xy](#). The usage is

```
runhaskell Runxy.hs <filename> <Int>
```

Alternatively, for faster execution time, you can compile Runxy.hs with

```
$ ghc --make Runxy.hs
```

and then run it with

```
$ ./Runxy <filename> <Int>
```

You can also use optimization options, such as

```
$ ghc --make -O2 Runxy.hs
```

We name the module `Main` even though the file is called `Runxy.hs`, so that we can compile to produce an executable as explained above:

```
module Main where

import System.Environment
```

We import the module `System.Environment` so that we can read the command line arguments with `getArgs`. We also need to import our own modules:

```
import AbstractSyntax
import Parser
import Interpreter
```

Because our little language doesn't have IO facilities, we use the variable `x` to hold the input and the variable `y` to hold the output. So this function creates a storage with the value `x` for the variable `"x"`, and with all other variables uninitialized, giving an error if we try to use them:

```
initialStorage :: Integer -> Storage
initialStorage x = update "x" x emptyStorage
```

Now, given a program in abstract syntax and a value for `x`, we run it with the above interpreter, and extract the value of the variable `"y"`.

runxy :: Program -> Integer -> Integer
runxy p x = m' "y"
 where
 m = initialStorage x
 m' = run p m

Finally, the `main` function reads the command line arguments with `getArgs`, then uses the module [Parser](#) to parse the file, and the module [Interpreter](#) to run the syntax tree produced by the parser on the given integer:

```
main :: IO()
main =
```

do

args <- getArgs

if length args == 2

then

do

concreteProgram <- readFile (args !! 0)

let abstractProgram = parseProgram concreteProgram

let x = read (args !! 1)

let y = runxy abstractProgram x

putStrLn (show y)

else

putStrLn "Usage: runhaskell Runxy.hs <filename> <Integer>"

Back to [table of contents](#)